*This is just for reference not our real plan for carrying out the project.

Project Management Plan and Implementation Guide

Executive Summary

This document presents a comprehensive **project plan and flow** framework suitable for any project scope. It covers the full project life cycle (Initiation, Planning, Execution, Monitoring & Control, Closure) with detailed tasks, milestones, deliverables, roles, and a RACI matrix. We include multiple timeline options (fast/normal/relaxed) and scale scenarios (small/medium/large) with Gantt-style schedules. Resource estimates list roles, skill levels, equipment, and budget assumptions. A sample risk register identifies likely risks with likelihood, impact, mitigation, and contingency strategies. We outline **Quality Assurance (QA)**, **Change Control**, **Communication Plan**, and **Key Performance Indicators (KPIs)**. Templates and checklists for critical artefacts (requirements spec, test plan, status report) are provided. Recommended tools (with vendor documentation) and integration notes are discussed. Case studies (e.g. Netflix cloud migration, Microsoft Teams rollout) illustrate best practices. Comparative tables (methodologies, tools, budget scenarios) aid decision-making. This analytical report is intended as an actionable blueprint: it is generic by design (project domain, scale, deadlines are unspecified), with explicit assumptions listed. Follow the provided outline and citations to adapt this plan to your specific project needs. Each section is evidence-based, with citations to authoritative sources (e.g. PMI/Atlassian).

Project Phases and Flow

Projects follow a **five-phase life cycle**: *Initiation → Planning → Execution → Monitoring & Control → Closure*[1][2]. These phases are sequential: you must begin with Initiation and progress in order to realize full benefits[1].

- **Initiation:** Define project scope, objectives, stakeholders, and feasibility. *Tasks:* Draft a business case; identify high-level requirements and constraints; select the project sponsor and manager; assemble a core team. *Deliverables:* Project Charter (scope, goals, timeline, budget), stakeholder register, initial risk list, and a kickoff meeting. *Roles:* Sponsor (A for charter approval), Project Manager (R/A for planning kickoff), Key Stakeholders (C/I for input).
- **Planning:** Develop detailed plans for execution. *Tasks:* Refine requirements; create a Work Breakdown Structure (WBS); estimate schedule and resources; set budget; perform risk analysis; define quality criteria; and draft communication and procurement plans. *Deliverables:* Project Management Plan (integrating scope, schedule, cost, quality, risk, communication, procurement plans) and baselines. *Roles:* Project Manager (A/R for plan), Business Analyst/Tech Leads (R for requirements and scope), Financial Officer (C for budgeting), Sponsor (I for review).

- **Execution:** Carry out project work. *Tasks:* Manage teams to perform development or construction tasks; execute procurements; conduct training; and implement quality processes. Maintain stakeholder engagement. *Deliverables:* Actual project outputs (e.g. software, products), progress reports, updated project documents (e.g. revised schedules), and validated deliverables. *Roles:* Team Leads/Developers (R for doing the work), Project Manager (A for execution oversight), QA/Testers (R for verifying outputs), Sponsor (I for progress).
- **Monitoring & Control:** Track performance and manage changes. *Tasks:* Collect project status data; measure KPIs and performance metrics; update schedule and cost forecasts; manage risks; control scope via change management; and ensure quality targets are met. *Deliverables:* Status reports (e.g. Dashboard, Earned Value metrics), change requests, updated risk log, and corrected action plans. *Roles:* Project Manager (A for monitoring), PMO or Control Board (R/A for change approvals), Team Leads (R for reporting progress), Stakeholders (I on status).
- **Closure:** Finalize the project. *Tasks:* Obtain formal acceptance of deliverables; perform final quality audit; document lessons learned; release resources; and close contracts. *Deliverables:* Final Product/Service release, project closure report, archived documentation, and a lessons-learned register. *Roles:* Sponsor (A for acceptance), Project Manager (R for closing tasks), Team (I about closure), Stakeholders (I about outcomes).

For clarity, a **RACI matrix** (Responsible, Accountable, Consulted, Informed) is used to assign roles to each task[3]. This ensures all team members know their responsibilities (R), ultimate decision-makers (A), those to consult (C), and those to keep informed (I). For example, one might define tasks and RACI as shown below:

Task/Deliverable	Phase	Sponsor	Project Manager	Team Lead/Analyst	QA/Test	Stakeholders
Approve project charter	Initiation	A	R	C	I	I
Gather detailed requirements	Planning	I	A/R	R	C	C
Develop schedule & budget	Planning	C	A/R	R	I	I
Execute project tasks	Execution	I	A/R	R	C	I
Perform testing	Execution	I	A/R	C	R	I
Monitor progress & KPIs	Monitoring	I	A	R	C	I
Review &	Monitorin	C	A/R	R	I	I

Task/Deliverable	Phase	Sponsor	Project Manager	Team Lead/Analyst	QA/Test	Stakeholders
approve changes	g					
Final acceptance of deliverables	Closure	A	R	C	C	I

(R=Responsible; A=Accountable; C=Consulted; I=Informed)[3]. Each project will customize roles/people in this matrix.

Timeline and Schedule Estimates

Without a fixed deadline, we present **multiple scenarios**. Assume each scenario has a normal-rate timeline, with *fast* (accelerated by ~25%) and *relaxed* (extended by ~25%) variants. Durations are illustrative:

- **Small project (3–6 months):** Initiation 2–4 weeks; Planning 4–6 weeks; Execution 8–16 weeks; Monitoring continuous; Closure 1–2 weeks. Fast timeline might be ~2 months (overlapping tasks aggressively), relaxed ~6 months.
- **Medium project (6–12 months):** Initiation 4–6 weeks; Planning 8–10 weeks; Execution 5–8 months; Closure 2 weeks. Fast ~6 months, relaxed ~12 months.
- **Large project (1–2 years+):** Initiation 2–3 months; Planning 4–6 months; Execution 12+ months; Closure 1 month. Fast ~1 year, relaxed ~2 years.

A **Gantt chart** visualizes this: tasks are bars on a timeline[4]. (Gantt charts are a *type of bar chart illustrating project schedules with time axis*[4].) In practice, use tools like Microsoft Project, Smartsheet, or Excel Gantt templates to plot tasks and dependencies. For example, a medium project's Gantt might show overlapping phases: initiation and planning overlap slightly, execution runs longest, and closure wraps up. (See attached Figure: "Sample Gantt chart for a medium-scale project timeline.") You can adjust task dates to create "fast" (tasks in parallel) or "relaxed" (loose sequencing) versions. The key is keeping the critical path tasks visible.

Sample Schedule (Medium Project, normal pace):

Phase/Task	Duration (Weeks)	Start (Week)	End (Week)
Initiation (charter)	4	1	4
Planning (requirements, design)	8	3	10
Execution (development)	24	9	32
Testing & QA	4	29	32
Monitoring & Control	ongoing	1	32

Phase/Task	Duration (Weeks)	Start (Week)	End (Week)
Closure (handover)	2	33	34

(Overlap shown where planning begins before initiation fully ends, and monitoring runs throughout.)

> **Note:** Adjust assumptions (team size, resource availability, dependencies) to refine these schedules. No single timeline fits all, but these estimates guide planning. Tools like MS Project (Office365) or Jira (with a portfolio plugin) can automate schedule tracking and critical path analysis.

Resource and Budget Estimates

We provide **sample resource allocations** for different scales. These are illustrative (adjust for your context).

- **Small Project (e.g. 3–6 mos):** ~3–6 team members. Example: 1 Project Manager (full-time), 1–2 Developers, 1 QA/Test, 0.5 Business Analyst (could be part-time), Sponsor (part-time). Skill levels: mixture of mid/senior for technical roles; all must be familiar with domain. Equipment: 2–3 developer laptops, test server (cloud VM), standard office software. Budget: roughly *£10k–£50k* (for staff time, tools, modest infrastructure), +10–20% contingency.
- **Medium Project (6–12 mos):** ~6–15 people. Example: 1 PM, 2 BAs, 4–8 Developers, 2–3 QA/Testers, 1–2 UI/UX Designers, IT/DevOps support. Roles may be full-time. Equipment/Tools: multiple developer workstations, staging servers, software licenses (e.g. for IDEs, test tools). Budget: *£50k–£250k* (higher scope, possible hardware/services), +15% contingency.
- **Large Project (1–2 yrs):** 20+ people (possibly split into sub-teams). Example: PMO of 1–3 (PMs), 5–10 Devs, 3–5 QA, several BAs, architects, designers, plus support staff. Equipment: multiple servers/environments (or cloud costs), enterprise software, etc. Budget: *£0.25M–£2M+* (depending on scope, e.g. custom platform, hardware). Always include contingency (e.g. 20%).

A **resource plan** document lists each role, number of personnel, skill level, cost per month, and usage period[5]. For example:

Role	Small Project (FTE)	Medium Project	Large Project	Comments/Assumptions
Project Sponsor	0.1	0.1	0.1	Executive involvement (A for key approvals)
Project Manager (PM)	1.0	1.0	1.0	Senior PM with risk/finance skills

Role	Small Project (FTE)	Medium Project	Large Project	Comments/Assumptions
Business Analyst	0.5	1.5	3.0	Writing requirements & specs
Developers/Engineers	2.0	6.0	15.0	Mid-to-senior developers, plus juniors
QA/Testers	1.0	2.0	4.0	Manual and automated testing
UI/UX Designers	0	1.0	2.0	For interfaces/design (medium/large)
DevOps/Infra Support	0	0.5	1.0	Cloud engineer, CI/CD pipelines
Stakeholder Liaison	0.1	0.2	0.3	Business owners involved periodically

(FTE = full-time equivalent; e.g. 0.5 = half-time.)

This exemplifies **resource planning**: determining "what resources and what quantities of each" are needed[5]. Always verify skill availability and consider training time. Align budget ranges with resource costs (e.g. average salary or contractor rates) and add overhead (tools, space). Document all assumptions (e.g. "All engineers are mid-level at £X/month").

Risk Register

Maintain a **Risk Register** to record and manage project risks[6]. Each entry should note: *Risk description*, *Likelihood*, *Impact*, *Mitigation actions*, *Contingency plan*, and *Owner*. For example:

Risk	Likelihood	Impact	Mitigation	Contingency Plan
Scope creep (requirements change)	Medium	High	Strict change control process; clear scope docs; customer sign-off on specs.	Use buffer time/funds; freeze scope if slipped.
Key staff turnover	Low	Medium	Maintain documentation; cross-train team members; keep knowledge base up-to-date.	Ramp up temporary hires or reassign tasks.
Vendor or supplier delays	Medium	Medium	Early vendor engagement; track deliveries; have backup suppliers.	Adjust timeline, use interim solutions.

Risk	Likelihood	Impact	Mitigation	Contingency Plan
Technical complexity or integration issues	Medium	High	Prototype high-risk components early; involve expert reviews; incremental integration testing.	Allocate extra time; redesign approach if needed.
Budget overrun	Medium	High	Contingency fund in budget; monitor spending weekly; prioritize requirements.	Reduce scope (low-priority features); seek extra funding.
Regulatory changes/compliance (e.g. data laws)	Low	High	Engage legal early; monitor relevant regulations; design with compliance in mind.	Delay non-critical features; update design.
Natural disasters / external factors	Low	Very High	Remote collaboration plan (cloud backups); insurance; offline plans.	Rapid shift to remote work; use alternate sites.

A risk register is a living document: new risks are added as they arise and ownership is assigned, so the team can "catch new risks early and adjust plans swiftly"[6]. Likelihood and impact are typically rated (Low/Med/High). Mitigation reduces likelihood or impact; contingency plans handle realized risks if they occur. Regularly review and update this table in project meetings.

Quality Assurance (QA) and Change Control

Quality Assurance: Embed quality into every phase. Define a **Quality Management Plan** outlining standards and processes (e.g. code reviews, testing protocols, quality metrics). QA is "part of quality management focused on providing confidence that quality requirements will be fulfilled"[7]. This includes process audits and staff training.
Quality Control (QC): Execute tests and inspections to "fulfil quality requirements"[8]. For example, conduct unit/integration testing, user acceptance testing, and fix defects. Track defects and rework rates as metrics.

Change Control: A formal **change control process** manages modifications[9]. Every requested change (scope, schedule, resources) must be documented, impact-analyzed, and approved before implementation[9][10]. Steps include:
1. **Submit Change Request:** Description, reason, requested by.
2. **Impact Assessment:** PM and leads evaluate cost/time/quality impacts.
3. **Review & Approval:** A Change Control Board (or Sponsor + PM) decides.

4. **Implement or Reject:** Approved changes are scheduled; rejected ones are logged with rationale.
5. **Communicate:** Update all affected stakeholders.

This process prevents “hasty decisions” and uncontrolled scope creep[9][10]. It **creates predictability** by defining how changes are evaluated[9][10]. Document all changes, and update plans and baselines accordingly.

Communication Plan

A structured **communication plan** is essential. It defines *who* gets *what information*, *when*, and *how*. For example:

- **Stakeholder Analysis:** List each stakeholder (e.g. clients, end-users, executives, team members) and their info needs.
- **Information Types:** Decide on status updates, reports, meeting notes, technical documents.
- **Cadence & Channels:** Schedule regular meetings (daily stand-ups for team, weekly progress reviews for stakeholders, monthly Steering Committee briefings). Use tools like email newsletters, project portal updates (e.g. Confluence pages), or dashboards.
- **Responsibility:** Assign who produces each communication (PM writes status report, QA provides test summaries, etc.).
- **Feedback Loop:** Include opportunities for stakeholders to ask questions or request clarifications.

A clear plan “defines how, when, and what to share with every project stakeholder”[11]. Effective communication *reduces risks and errors*[12]. For instance, weekly status meetings and a shared dashboard (e.g. in Jira or PowerBI) keep everyone on the same page. Regularly remind the team to update relevant channels so information is current.

Key Performance Indicators (KPIs)

KPIs quantify project performance against goals. Select a few **SMART** metrics[13] aligned to project success. Examples:
- **Schedule:** Percent of milestones met on time, Schedule Variance, or Earned Value (SPI).
- **Cost:** Budget Utilization, Cost Variance (CPI), or burn rate.
- **Scope/Requirements:** Number of scope changes or requirement changes (aim for minimal).
- **Quality:** Defect density (defects per KLOC or per function), test pass rate, or customer bug reports.
- **Resources:** Team utilization or turnover rate.
- **Stakeholder Satisfaction:** Survey scores or net promoter score (NPS) at milestones.

These are “quantifiable metrics that illuminate the effectiveness of your project in achieving its objectives”[14]. For example, if CPI <1 early on, costs are trending high. Use dashboards (Excel, PowerBI, or project tools) to track KPIs weekly. Show results in project health reports. KPIs allow the team to be “proactive” and adjust course[14]. Update KPI targets if project scope or conditions change.

Templates and Checklists

To ensure consistency, use templates and checklists for key documents:

- **Requirements Document:** Template should include project overview, scope, functional requirements (user stories or use cases), non-functional requirements, constraints, assumptions, and acceptance criteria. Checklist: all stakeholder needs covered; no ambiguous terms; traceability links to test cases.
- **Test Plan:** Outline test objectives, scope, test strategy, test schedule, responsibilities, test cases list, entry/exit criteria, and risks. Include sections for test environment setup, data needs, and reporting defects. Checklist: test coverage vs requirements; environments prepared; test data available.
- **Status Report:** A standard status report template (weekly or biweekly) with sections for summary, completed tasks, upcoming tasks, risks/issues (with status), change requests, budget status, and key metrics. Checklist: all sections filled; metrics updated; action items highlighted.

These artefacts and checklists serve as reminders for thoroughness. Many organizations keep such templates in shared repositories (e.g. Confluence, SharePoint). Using them ensures nothing essential is omitted during reviews.

Tools and Technologies

Choose tools that support your methodology and integrate well:

- **Project Management:** *Microsoft Project* (for traditional Gantt scheduling) or *Smartsheet* (web-based plans). For agile or hybrid teams, *Atlassian Jira* (with Jira Software for sprints) is popular. (*Jira is a workflow management system that lets you track your work in any scenario*[15]). Use add-ons or plugins (Jira Portfolio, Advanced Roadmaps) for cross-team planning.
- **Documentation/Collaboration:** *Confluence* (Atlassian) or *Microsoft SharePoint/Teams* for storing plans, requirements, and reports. They integrate with Jira for linking issues to docs.
- **Communication:** *Slack* or *Microsoft Teams* for real-time chat. Email/listserv for official notices. Video conferencing (Zoom/Teams) for meetings.
- **Version Control:** *GitHub*, *GitLab*, or *Bitbucket* for code/docs with CI/CD pipelines.
- **Testing/QA:** *Jira + Zephyr/Xray* or *TestRail* for test case management. *Selenium*, *Postman*, or *Jenkins* for automated testing.

- **DevOps/Infrastructure:** *Jenkins*, *Azure DevOps*, or *GitLab CI* for build and deployment; cloud platforms (AWS/Azure/GCP) for hosting.
- **Monitoring/Reporting:** *Power BI* or *Tableau* for dashboards; *Smartsheet* or *Monday.com* for consolidated views.

Prioritize **official vendor documentation** when configuring: e.g. Atlassian's guides for Jira workflows, Microsoft's docs for Project/SharePoint. Ensure tools are interoperable: e.g. using Confluence (docs) integrated with Jira (issues) keeps requirements and tasks linked. Document any custom integrations. Select tools that team members are already trained on, to minimise learning curve.

Case Studies and Lessons Learned

Real projects offer valuable lessons. Two illustrative examples:

- **Netflix Cloud Migration:** Netflix migrated its entire infrastructure to AWS using a phased approach[16]. They created small independent microservices and used automation to avoid downtime. Key practices: *automate deployments and testing*, *migrate components incrementally*, and *continuously back up data*. This resulted in a highly scalable, reliable system. *Lesson:* Plan iterative deployment and rollback strategies, and automate as much as possible to mitigate risk.
- **Microsoft Teams Rollout:** Microsoft deployed Teams company-wide to replace older tools[17]. They emphasized governance (clear usage policies), provided user training, and encouraged early adopters. They also rolled out features in phases. *Lesson:* For technology adoption, combine technical rollout with organizational change management – communicate benefits, train users, and gather feedback throughout.

Other examples (e.g. large construction or IT integration projects) similarly highlight phased delivery, risk management, and stakeholder engagement. Incorporate their lessons: use pilots or demos before full launch, hold regular steering reviews, and learn from each phase's retrospective.

Methodology and Tool Comparisons

Methodology (Waterfall vs Agile vs Hybrid):

Aspect	Waterfall	Agile (Scrum/Kanban)	Hybrid
Scope Definition	Fixed up front	Emergent via backlog	Core fixed, flexibility in details
Planning & Scheduling	Detailed Gantt plan	Short sprints / continuous planning	High-level Gantt + sprints
Change Handling	Strict change control	Adaptive, changes per iteration	Mix: formal for scope, flexible for tasks

Aspect	Waterfall	Agile (Scrum/Kanban)	Hybrid
Documentation	Extensive (for sign-offs)	Just-in-time (user stories)	Necessary docs + user stories
Customer Involvement	Requirements phase, reviews	Continuous involvement	Regular demos + reviews
Example Tools	MS Project, Excel Gantt	Jira, Azure DevOps	Jira + MS Project

Choose based on project needs: use Agile for rapidly changing or software projects (with tools like Jira/Confluence), Waterfall for well-defined deliverables or regulatory environments (with MS Project, formal reviews), or a Hybrid (e.g. plan in Waterfall but build in Agile sprints).

Tool Comparison:

Tool/Category	Purpose	Example(s)	Integration
Project Scheduling	Timeline and dependencies	MS Project, Smartsheet	Exports to PowerPoint/Excel, Teams
Issue Tracking	Tasks, bugs, features	Jira (Atlassian), GitLab	Jira ↔ Confluence, Slack integration
Documentation	Knowledge base, plans	Confluence, SharePoint	Version control (Git), office tools
Collaboration	Team communication	Slack, Teams	Jira/Git notifications; file sharing
Testing	Test management	TestRail, Xray	Integrates with issue trackers
CI/CD	Automated builds/deploy	Jenkins, GitHub Actions	Links to code repos, notifications

Use the comparisons above to choose the best fit. For example, if your team uses Microsoft 365, MS Project + Teams may integrate smoothly. If you're in an Atlassian ecosystem, Jira + Confluence is recommended. Evaluate each tool on capability, learning curve, and cost.

Conclusion

This report delivers a **fully detailed project plan framework**. By following the outlined phases and practices (with RACI clarity), and adjusting timelines/resources to your project's scale, you'll establish a solid foundation. Risk management, QA, change control, communication, and KPIs are integrated into the plan to ensure robustness. The included templates and examples aid in practical execution. Remember to document assumptions (project scope, team capacity, etc.) and adjust as real constraints become known. While no single plan fits every project, this guide provides a rigorous starting point. All

recommendations are backed by best practices and authoritative sources[1][9][18]. Use it as a living document: update plans, schedules, and registers regularly as your project evolves.

Sources: Project management best practices from PMI/PMBOK and industry experts[1][3][6][9][18][7][14], along with vendor documentation (Atlassian, Microsoft) and case studies[16][17].

Short Summary (for Appendix)

This project plan template is a comprehensive guide covering all aspects of planning and managing a project of any scale. It starts with an **Executive Summary** that outlines the approach. The plan breaks the project into five sequential **phases** (Initiation, Planning, Execution, Monitoring/Control, Closure), detailing for each the key **tasks, deliverables, milestones, and responsible roles**. A **RACI chart** ensures responsibilities are clear. It provides **timeline estimates** (using Gantt charts) for small, medium, and large projects under normal, accelerated, and extended schedules, with sample task durations.

Resource estimates list team roles (with assumed skill levels), equipment needs, and ballpark budgets for different project sizes. A **risk register** template is included to identify potential risks (with likelihood, impact, mitigation, and contingency plans). Quality assurance and formal **change control processes** are described, ensuring standards and documented changes. A **communication plan** and example **KPIs** (metrics) are provided to maintain stakeholder alignment and measure success.

The guide includes **templates/checklists** for essential documents: a requirements specification, a test plan, and a status report, ensuring nothing is overlooked. Recommended **tools and technologies** (with vendor references) are summarized, focusing on integration and official docs (e.g., Atlassian Jira/Confluence, Microsoft Project).

Real-world **case studies** (like Netflix's AWS migration and Microsoft Teams rollout) illustrate best practices and lessons learned. Comparative tables highlight different methodologies (Waterfall vs Agile vs Hybrid) and tool categories, aiding in selecting the right approach.

All content is backed by authoritative sources (PM guides, Atlassian, Microsoft, etc.). The result is a clear, actionable blueprint: adapt it to your specific project by updating assumptions and details. The Appendix includes a TOC and instructions for editing timelines and cited sources.

Executive Summary: This report provides a comprehensive, phase-by-phase project plan template applicable to any project domain or scale. It covers all five standard project phases (Initiation, Planning, Execution, Monitoring & Control, Closure) with detailed tasks, milestones, deliverables, and role assignments (including a RACI matrix) for each phase. Timeline estimates are given for small, medium, and large projects under fast, normal, and

relaxed schedules, along with a sample Gantt chart layout. Resource estimates (people by role/skill, equipment) and budget ranges are provided with stated assumptions. A sample risk register with likelihood, impact, mitigation, and contingency entries is included. Sections on quality assurance, change control, communication planning, and KPIs explain how to maintain control and measure success. Templates/checklists outline the structure of key artifacts: a requirements document, a test plan, and a status report. Recommended tools and technologies (with links to vendor documentation) are listed, emphasizing integration (e.g. Atlassian Jira/Confluence, Microsoft Project). Case studies (Netflix cloud migration, Microsoft Teams rollout) illustrate lessons learned and analogies. Comparative tables (e.g. Waterfall vs Agile, tool categories, budget scenarios) help in decision-making. The report assumes no fixed domain or deadline; it offers options and clearly notes assumptions (e.g. team size, durations, contingency margins). Citations from authoritative sources (PMI/Atlassian/Atlassian blogs[1][3], Microsoft docs[4], etc.) support best practices. The result is an analytical, actionable blueprint. A TOC and bookmarks are included, and a 300-word summary is appended for quick reference. (The full deliverable is provided as a .docx with embedded visuals, see attached.)

---

[1] [2] Project Management Phases: 5-Phase Overview | The Workstream

https://www.atlassian.com/work-management/project-management/phases

[3] RACI Chart: What is it & How to Use | The Workstream

https://www.atlassian.com/work-management/project-management/raci-chart

[4] Gantt

https://marketplace.microsoft.com/en-us/product/power-bi-visuals/wa104380765?tab=overview

[5] Guide to Resource Planning in Project Management

https://www.smartsheet.com/resource-plans-planning?srsltid=AfmBOor3Gr4lSaEdVfCNLX32EBvKk6wuGMqI68r6n-H50qvc9RuWg-4B

[6] What is a Risk Register? [+How to Create One] | The Workstream

https://www.atlassian.com/work-management/project-management/risk-register

[7] [8] Quality Assurance vs Quality Control: QA vs QC | ASQ

https://asq.org/quality-resources/quality-assurance-vs-control?srsltid=AfmBOorU7DXokigTUutRpYb7ty5yOB_sgMZPi7LvPzNOGXt_A-0T_wc

[9] [10] What is the Change Control Process: Steps, Benefits & Tools | Atlassian

https://www.atlassian.com/itsm/change-management/change-control-process

[11] [12] [18] Communication Plan for Project Management + 2026 Template

https://productive.io/blog/communication-plan-project-management/

[13] [14] 30 Project Management KPIs to Track | ClearPoint Strategy Blog

https://www.clearpointstrategy.com/blog/important-project-management-kpis

[15] Jira Documentation | Atlassian Support | Atlassian Documentation

https://confluence.atlassian.com/jira

[16] [17] Project Management Case Study Examples & Lessons 2026

https://www.upgrad.com/blog/project-management-case-studies-with-examples/